

## Unit- 5

### Basic Architecture of an IoT-based Embedded System

An **IoT-based Embedded System** is a combination of **hardware, software, and communication technologies** designed to connect devices to the internet, collect data, and make intelligent decisions.

It integrates **embedded systems** (for sensing, processing, and control) with **IoT technologies** (for connectivity and cloud integration).

---

#### 1. Overview

- **IoT (Internet of Things)** connects physical objects (“things”) to the internet to collect, exchange, and act on data.
  - **Embedded systems** provide the intelligence to these devices — they sense, process, and control operations.
  - Together, they form a **smart system** capable of automation, remote control, and analytics.
- 

#### 2. Main Layers of IoT-Based Embedded Architecture

The architecture typically consists of **five layers**:

##### *(a) Perception Layer (Sensing Layer)*

- **Function:** Detects and gathers information from the physical environment.
  - **Components:**
    - Sensors: Temperature, humidity, pressure, motion, light, etc.
    - Actuators: Motors, valves, relays that perform physical actions.
    - Embedded microcontrollers for local data acquisition.
  - **Example:** A temperature sensor reading room temperature for a smart thermostat.
- 

##### *(b) Network Layer (Communication Layer)*

- **Function:** Transfers the collected data to other devices or cloud servers.
  - **Components:**
    - Communication modules: Wi-Fi, Bluetooth, ZigBee, LoRa, 4G/5G, Ethernet.
    - Gateways or routers for data routing and protocol translation.
  - **Example:** Wi-Fi module sending sensor data from an Arduino or Raspberry Pi to the cloud.
-

### ***(c) Edge or Processing Layer***

- **Function:** Performs local data processing, filtering, and decision-making near the data source.
  - **Components:**
    - Microcontrollers / Microprocessors: ARM Cortex, Raspberry Pi, ESP32, etc.
    - Real-Time Operating Systems (RTOS) or embedded OS.
    - Edge analytics (e.g., preprocessing sensor data to reduce bandwidth).
  - **Example:** Microcontroller decides to turn on a fan when the temperature exceeds a threshold.
- 

### ***(d) Cloud Layer (Middleware or Data Management Layer)***

- **Function:** Centralized data storage, analytics, and decision-making.
  - **Components:**
    - Cloud platforms: AWS IoT, Azure IoT Hub, Google Cloud IoT, ThingSpeak, etc.
    - Databases for storing large-scale IoT data.
    - Machine Learning models for predictive analysis.
  - **Example:** Cloud dashboard analyzing temperature trends over time.
- 

### ***(e) Application Layer***

- **Function:** Provides user interaction and visualization of IoT data.
  - **Components:**
    - Web and mobile apps, dashboards, or APIs.
    - Enables remote control, notifications, and data visualization.
  - **Example:** A smartphone app that shows home temperature and allows turning on/off appliances remotely.
- 

## **3. Supporting Components**

### ***(a) Power Management***

- IoT devices often use **low-power microcontrollers** and **energy harvesting** (solar, battery) to extend lifetime.

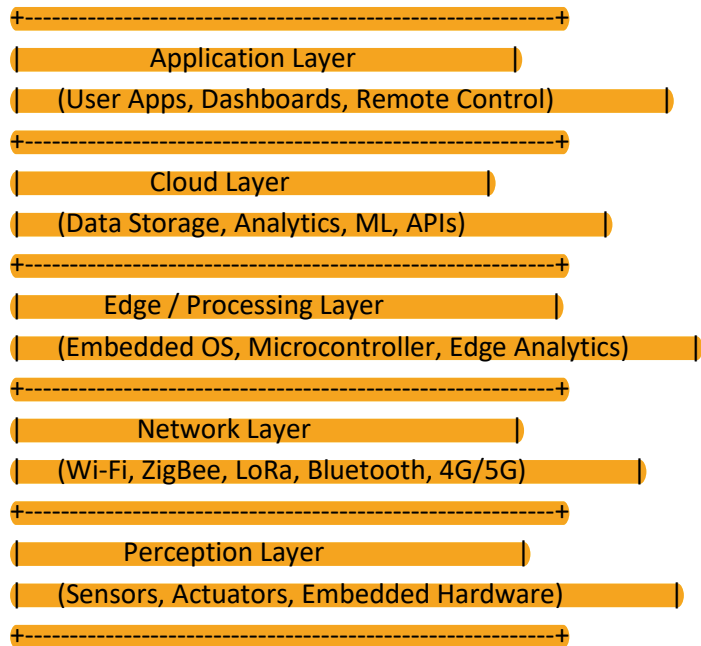
### ***(b) Security Layer***

- Provides **data confidentiality, integrity, and authentication.**
- Uses techniques like:
  - Encryption (AES, RSA)
  - Secure boot
  - Access control and firmware updates.

### (c) Device Management

- Monitors device health, firmware, and performance remotely.
  - Supports **Over-the-Air (OTA)** updates.
- 

### 4. Summary Diagram (Textual View)



### 5. Example: Smart Home Temperature System

- **Sensors:** Temperature sensor (e.g., DHT11)
  - **Processor:** ESP32 microcontroller
  - **Network:** Wi-Fi connectivity
  - **Cloud:** ThingSpeak for storing and analyzing temperature data
  - **Application:** Mobile app to display readings and control air conditioning
- 

### 6. Key Features of IoT Embedded Systems

- **Interconnectivity:** Devices communicate through standard protocols.
- **Intelligence:** Decision-making via embedded or cloud-based processing.
- **Scalability:** Supports adding new devices easily.
- **Automation:** Performs tasks without human intervention.
- **Real-time operation:** Processes data and reacts promptly.

# Embedded Hardware for IoT Applications

---

## □ 1. Introduction

- **Embedded hardware** is the physical foundation of any IoT system.
  - It consists of **microcontrollers, sensors, actuators, communication modules, and power units** integrated into a small, intelligent device.
  - These components together enable **data sensing, processing, communication, and control** in IoT applications.
- 

## □ 2. Major Components of Embedded IoT Hardware

### (a) Sensors

- **Function:** Sense or detect physical parameters from the environment and convert them into electrical signals.
  - **Examples:**
    - Temperature – DHT11, LM35
    - Humidity – DHT22
    - Motion – PIR Sensor
    - Pressure – BMP180
    - Light – LDR (Light Dependent Resistor)
  - **Role in IoT:** Collect raw environmental data for processing and decision-making.
- 

### (b) Actuators

- **Function:** Perform an action based on the controller's command (convert electrical signals into physical motion).
  - **Examples:**
    - Motors (servo, DC, stepper)
    - Valves, buzzers, LEDs, relays
  - **Use Case:** Controlling doors, fans, alarms, or robotic movement in IoT applications.
- 

### (c) Microcontrollers and Microprocessors

- **Function:** Act as the **brain** of the IoT system — process sensor data, run algorithms, and control outputs.
- **Key Differences:**
  - **Microcontroller (MCU):** Single chip with CPU, RAM, ROM, and I/O ports — ideal for low-power IoT devices.

- **Microprocessor (MPU):** More powerful; used in high-end IoT devices needing multitasking and OS (e.g., Linux).
- **Common IoT MCUs/MPUs:**

| Board/Chip               | Features                      | Common Use                                  |
|--------------------------|-------------------------------|---------------------------------------------|
| Arduino UNO (ATmega328P) | Simple, low-cost, open-source | Hobby & educational IoT projects            |
| ESP8266 / ESP32          | Built-in Wi-Fi/Bluetooth      | Smart home, IoT nodes                       |
| Raspberry Pi (ARM-based) | Mini computer with Linux      | Edge computing, AI, and camera applications |
| STM32 (ARM Cortex)       | Low power, industrial-grade   | Automotive, robotics                        |
| BeagleBone Black         | High I/O capability           | Industrial control systems                  |

#### (d) Communication Modules

- Enable **connectivity** between IoT devices and networks.
- **Types and Examples:**
  - **Wi-Fi:** ESP8266, ESP32, NodeMCU
  - **Bluetooth / BLE:** HC-05, nRF52832
  - **ZigBee:** XBee modules for mesh networks
  - **LoRa / LoRaWAN:** Long-range, low-power modules (e.g., SX1278)
  - **Cellular (4G/5G):** SIM800L, SIM900 for GSM/GPRS
  - **Ethernet:** ENC28J60 for wired IoT nodes
- **Selection Criteria:** Depends on range, power, data rate, and environment.

---

#### (e) Power Management Unit

- **Function:** Supplies and regulates power for all components.
- **Types:**
  - **Battery-powered:** Rechargeable Li-ion, Li-Po, or AA cells.
  - **Energy harvesting:** Solar panels or kinetic energy sources.
  - **Voltage regulators:** LM7805, AMS1117 to maintain constant voltage.
- **Power optimization techniques:**
  - Sleep modes
  - Duty cycling
  - Low-power MCUs

---

#### (f) Memory and Storage

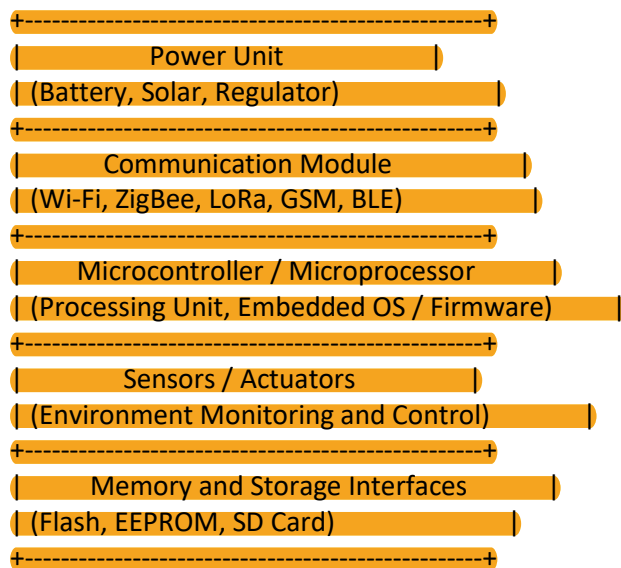
- **Function:** Store firmware, sensor data, and configuration details.
- **Types:**
  - **ROM / Flash memory:** For program code (firmware).
  - **RAM:** For temporary processing data.
  - **EEPROM / SD card:** For data logging and long-term storage.

- **Example:** ESP32 supports internal flash; Raspberry Pi uses SD cards.

### (g) Interface and I/O Components

- Provide connections for sensors, actuators, and external systems.
- **Examples:**
  - GPIO (General Purpose I/O) pins
  - ADC/DAC (Analog to Digital / Digital to Analog Converters)
  - Communication buses:
    - I<sup>2</sup>C (Inter-Integrated Circuit)
    - SPI (Serial Peripheral Interface)
    - UART (Universal Asynchronous Receiver Transmitter)
    - CAN (Controller Area Network – used in automotive IoT)

## 3. Typical Architecture of Embedded IoT Hardware



### 4. Examples of IoT Hardware Platforms

| Platform           | Key Features                                      | Typical Use                      |
|--------------------|---------------------------------------------------|----------------------------------|
| Arduino UNO / Nano | Open-source, easy to program, 14 digital I/O pins | Beginner IoT projects            |
| NodeMCU (ESP8266)  | Wi-Fi enabled, supports MQTT/HTTP                 | Smart home automation            |
| ESP32              | Dual-core MCU, BLE + Wi-Fi                        | Wearable and real-time IoT       |
| Raspberry Pi 4     | Full OS (Linux), USB, HDMI, Ethernet              | Edge computing, camera-based IoT |
| STM32 Nucleo       | ARM Cortex-M series                               | Industrial IoT and robotics      |
| BeagleBone Black   | Linux support, high-speed I/O                     | Real-time industrial systems     |

|                           |                                     |                         |
|---------------------------|-------------------------------------|-------------------------|
| Intel Edison /<br>Galileo | High-performance IoT gateway boards | Cloud-connected devices |
|---------------------------|-------------------------------------|-------------------------|

## 5. Supporting Hardware for IoT

- **Gateways:** Bridge local devices with cloud networks. (e.g., Raspberry Pi as IoT gateway)
- **Edge Devices:** Perform local analytics before data upload.
- **Cloud Integration Hardware:** Supports MQTT/HTTP protocols for cloud data transfer.

## 6. Example: Smart Agriculture IoT Node

| Component     | Example                                     | Function                            |
|---------------|---------------------------------------------|-------------------------------------|
| MCU           | ESP32                                       | Controls system & connects to Wi-Fi |
| Sensors       | DHT11 (temp/humidity), Soil moisture sensor | Measures environment                |
| Actuators     | Water pump                                  | Controls irrigation                 |
| Power         | Solar panel + Li-ion battery                | Continuous energy supply            |
| Communication | Wi-Fi                                       | Data upload to ThingSpeak / AWS IoT |

## 7. Summary

- Embedded hardware is the **core enabler** of IoT systems.
- Selection depends on **application type, power requirements, communication range, and cost.**
- A well-designed IoT hardware platform ensures **efficient sensing, processing, and communication** for real-world automation.

**Arduino and Raspberry-pi notes (separate)---**

# ARM Development Board

## 1. Introduction

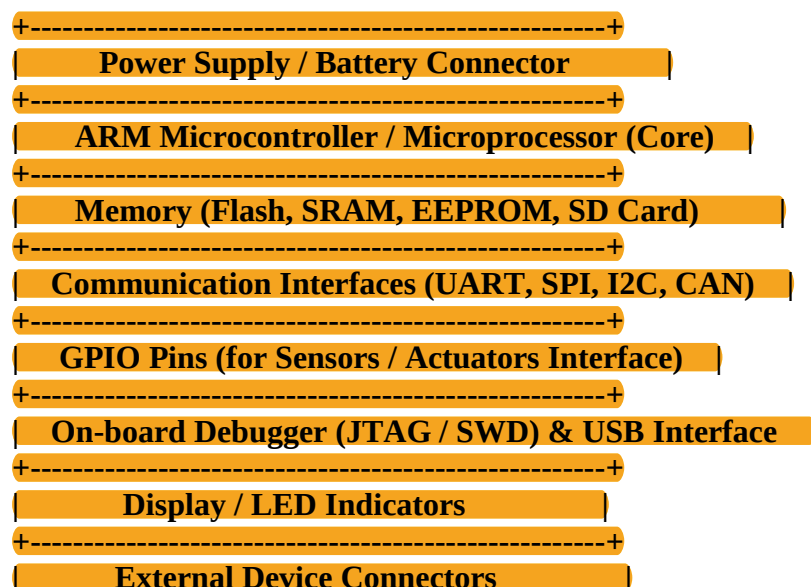
- **ARM (Advanced RISC Machine)** is a family of **RISC-based microprocessors** widely used in embedded and IoT systems.
  - **ARM Development Boards** are hardware platforms built around ARM microcontrollers or processors.
  - They are used for **developing, testing, and deploying** embedded and IoT applications.
  - ARM-based boards are known for being **power-efficient, compact, and cost-effective**.
- 

## 2. Why ARM for Embedded and IoT Systems

- **RISC Architecture:** Uses a simplified instruction set → faster and energy-efficient.
  - **Low Power Consumption:** Ideal for battery-operated IoT devices.
  - **Scalability:** Available in different series (Cortex-M, Cortex-A, Cortex-R) for different applications.
  - **Wide Ecosystem:** Supported by major vendors (STMicroelectronics, NXP, Texas Instruments, etc.).
  - **Rich Development Support:** Tools like **Keil uVision, STM32CubeIDE, ARM Mbed Studio, and Arduino IDE**.
- 

## 3. Basic Architecture of ARM Development Board

A typical ARM development board consists of the following key components:





#### 4. Major Components and Their Functions

| Component                    | Function                                                        |
|------------------------------|-----------------------------------------------------------------|
| ARM Processor Core           | Executes program instructions (Cortex-M, Cortex-A series).      |
| Memory (Flash, SRAM)         | Stores program code and runtime data.                           |
| Clock & Reset Circuit        | Provides stable timing signals and initializes MCU.             |
| Power Supply Unit            | Converts input voltage (USB, DC) into regulated levels for MCU. |
| I/O Ports (GPIO)             | Interface with sensors, actuators, and peripherals.             |
| Communication Interfaces     | UART, SPI, I2C, CAN, USB, Ethernet, etc.                        |
| Debug Interface (JTAG/SWD)   | Used for program uploading and debugging.                       |
| Connectors / Expansion Ports | Used to add shields, modules, or external devices.              |
| Indicator LEDs               | Show power, status, and communication activity.                 |

## 5. Popular ARM-Based Development Boards

### (a) STM32 Nucleo Board (STMicroelectronics)

- Based on **ARM Cortex-M** series.
- Supports **Arduino-compatible headers**.
- Integrated **ST-LINK/V2-1 debugger**.
- Used for **IoT, robotics, and control systems**.

### (b) LPC2148 Development Board (NXP)

- Based on **ARM7TDMI-S** core.
- Features **USB 2.0, UART, SPI, ADC, PWM, and GPIOs**.
- Commonly used in **academic and industrial embedded projects**.

### (c) Raspberry Pi (ARM Cortex-A Series)

- Acts as a **single-board computer (SBC)**.
- Runs **Linux-based OS**.
- Supports **Ethernet, Wi-Fi, USB, HDMI**.
- Used for **IoT gateways, AI edge computing, and prototyping**.

### (d) BeagleBone Black

- Based on **ARM Cortex-A8** processor.
- Includes **Ethernet, HDMI, GPIOs, and Linux OS**.
- Excellent for **real-time industrial applications**.

### (e) Arduino Due

- Based on **Atmel SAM3X8E ARM Cortex-M3** processor.
  - Fully compatible with Arduino IDE.
  - 84 MHz clock, 96 KB SRAM, 512 KB Flash.
- 

## 6. Typical Features of an ARM Development Board

- 32-bit or 64-bit RISC CPU
  - Multiple communication interfaces
  - Analog and digital I/O support
  - Timers, PWM, and ADC converters
  - Support for real-time operating systems (RTOS)
  - Low-power modes for IoT
  - Debugging and programming ports
- 

## 7. Applications of ARM Development Boards

- Smart home automation
  - Robotics and drones
  - Industrial control systems
  - Medical devices
  - Smart agriculture sensors
  - IoT data collection nodes
  - Wearable devices and consumer electronics
- 

## 8. Example: LPC2148 ARM7 Development Board

| Feature                  | Specification                  |
|--------------------------|--------------------------------|
| Processor                | ARM7TDMI-S core                |
| Clock Speed              | 60 MHz                         |
| Memory                   | 512 KB Flash, 32 KB SRAM       |
| Communication Interfaces | UART, SPI, I2C, USB            |
| GPIO Pins                | 45                             |
| ADC Channels             | 14 (10-bit)                    |
| PWM Channels             | 6                              |
| Operating Voltage        | 3.3V                           |
| Programming Port         | USB / JTAG                     |
| Development Tools        | Keil $\mu$ Vision, Flash Magic |

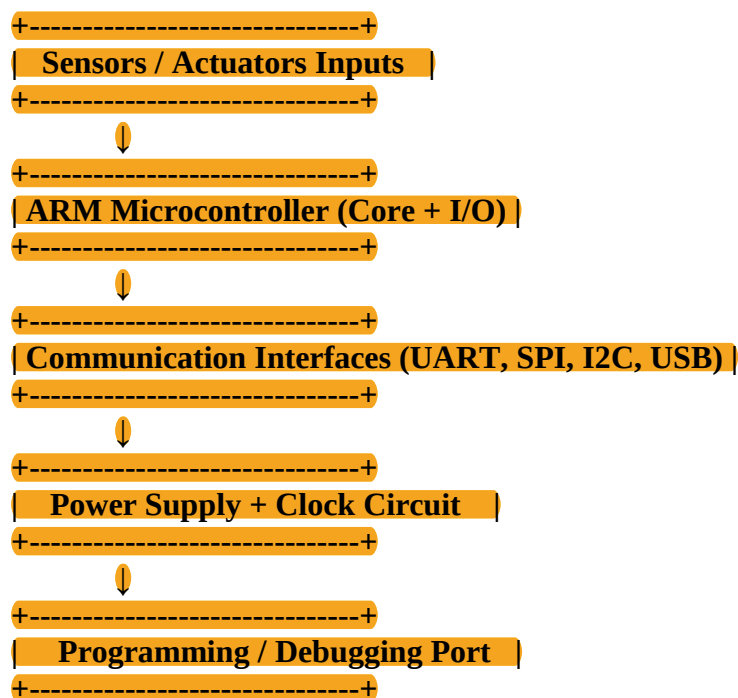
## 9. Working Flow of ARM Development Board

1. **Write Program** – in C/C++ using IDE (e.g., Keil, STM32CubeIDE).
  2. **Compile & Build** – generates a hex/bin file.
  3. **Upload Program** – to the board using JTAG/SWD/USB.
  4. **Execute Program** – the MCU runs the code and interacts with connected devices.
  5. **Debug/Test** – monitor variables, I/O, and communication through IDE tools.
- 

## 10. Advantages

- High performance with low power usage.
  - Scalable from small microcontrollers to high-end processors.
  - Rich ecosystem and vendor support.
  - Real-time performance with RTOS integration.
  - Cost-effective and widely available.
- 

## 11. Example Diagram (Text View)



## 12. Summary

- ARM development boards form the **core hardware** of embedded and IoT projects.
- They offer a **flexible, efficient, and scalable** platform for developing real-world smart systems.

- With various models like **STM32**, **LPC2148**, **Raspberry Pi**, and **BeagleBone**, ARM boards cater to applications from **sensor nodes** to **IoT gateways**.

## IoT Cloud Platform and IoT Client Applications on Mobile Phones

### 1. Introduction

The **Internet of Things (IoT)** involves connecting smart devices to collect, store, and analyze data through the internet.

Two key components in any IoT ecosystem are:

1. **IoT Cloud Platforms** – provide data storage, analytics, and device management.
2. **IoT Client Applications** – mobile or web apps that allow users to monitor and control IoT devices.

Together, they form the **backend and frontend** of an IoT system.

---

### 2. IoT Cloud Platform

#### Definition

An **IoT Cloud Platform** is an **online infrastructure** that enables:

- Data collection from IoT devices
- Data storage and visualization
- Real-time analytics
- Device management and security
- Integration with other applications (via APIs)

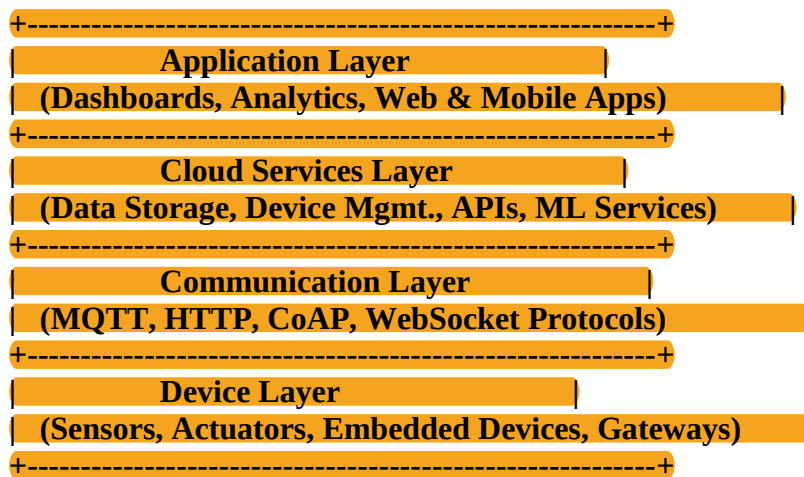
It acts as a **bridge between IoT devices and end-user applications**.

### 3. Key Functions of an IoT Cloud Platform

| Function | Description |
|----------|-------------|
|----------|-------------|

|                                             |                                                                                      |
|---------------------------------------------|--------------------------------------------------------------------------------------|
| <b>Device Connectivity &amp; Management</b> | Registers and manages multiple IoT devices using protocols like MQTT, HTTP, or CoAP. |
| <b>Data Collection &amp; Storage</b>        | Collects sensor data and stores it securely in the cloud database.                   |
| <b>Data Processing &amp; Analytics</b>      | Performs filtering, aggregation, or machine learning on IoT data.                    |
| <b>Visualization</b>                        | Provides dashboards, graphs, and real-time monitoring tools.                         |
| <b>Security &amp; Authentication</b>        | Ensures safe communication via encryption, tokens, and access control.               |
| <b>APIs &amp; Integration</b>               | Provides RESTful APIs to connect with web/mobile apps and enterprise systems.        |

## 4. Architecture of IoT Cloud Platform



## 5. Popular IoT Cloud Platforms

| Platform                         | Provider            | Features                                                             |
|----------------------------------|---------------------|----------------------------------------------------------------------|
| <b>AWS IoT Core</b>              | Amazon Web Services | Secure, scalable, supports MQTT/HTTP, integrates with AI/ML services |
| <b>Google Cloud IoT Core</b>     | Google              | Big data analytics, visualization, machine learning                  |
| <b>Microsoft Azure IoT Hub</b>   | Microsoft           | Device-to-cloud and cloud-to-device messaging, strong security       |
| <b>IBM Watson IoT Platform</b>   | IBM                 | Real-time data analytics and cognitive services                      |
| <b>ThingSpeak</b>                | MathWorks           | Simple open-source platform, great for academic IoT projects         |
| <b>Adafruit IO / Blynk Cloud</b> | Community Platforms | Ideal for prototyping and educational use                            |

## 6. Common Communication Protocols Used

- **MQTT (Message Queuing Telemetry Transport)** – Lightweight, ideal for low-power IoT devices.
  - **HTTP/HTTPS** – Standard web protocol for communication.
  - **CoAP (Constrained Application Protocol)** – Designed for low-power, resource-constrained devices.
  - **WebSocket** – Enables continuous bi-directional communication.
- 

## 7. Example: Data Flow in an IoT Cloud Platform

1. Sensor (e.g., temperature) collects data.
  2. Microcontroller (ESP32) sends data via MQTT to cloud.
  3. Cloud platform stores data and performs analytics.
  4. Cloud dashboard or mobile app retrieves data via APIs.
  5. User views real-time readings or sends control commands back.
- 

## 8. IoT Client Applications on Mobile Phones

### Definition

IoT client applications are **mobile apps** that allow users to:

- Monitor IoT device status and sensor data.
- Control connected devices remotely.
- Receive alerts, notifications, or analytics results.

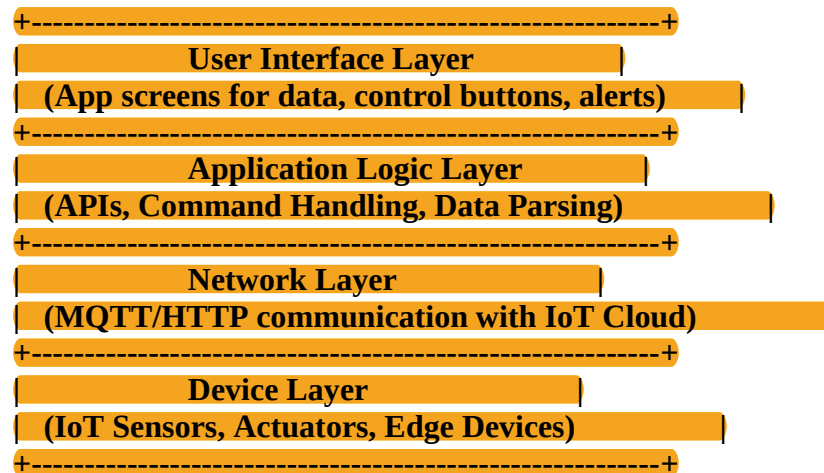
They are the **user interface** for interacting with IoT systems.

---

## 9. Key Functions of IoT Mobile Applications

| Function             | Description                                                         |
|----------------------|---------------------------------------------------------------------|
| Device Monitoring    | Displays real-time data (temperature, humidity, energy usage, etc.) |
| Device Control       | Allows remote control actions (turn ON/OFF, adjust parameters)      |
| Notifications/Alerts | Sends warnings for critical conditions (e.g., high temperature)     |
| Data Visualization   | Graphs, charts, and logs for trend analysis                         |
| User Authentication  | Ensures only authorized users can access or control devices         |
| Integration          | Works with IoT cloud platforms via REST APIs or MQTT brokers        |

## 10. Architecture of IoT Mobile Application



## 11. Technologies Used for IoT Mobile Apps

- **Android:** Kotlin, Java, Flutter, or React Native.
- **iOS:** Swift, Flutter.
- **Cross-platform frameworks:** React Native, Ionic, Xamarin.
- **Communication Libraries:**
  - MQTT Client (e.g., Eclipse Paho)
  - HTTP/REST APIs
  - WebSocket for real-time communication.

## 12. Example IoT Mobile Apps

| App                     | Purpose                                      | Connected Platform |
|-------------------------|----------------------------------------------|--------------------|
| Blynk IoT App           | Build custom dashboards with buttons, graphs | Blynk Cloud        |
| ThingView App           | Monitor ThingSpeak data                      | ThingSpeak Cloud   |
| AWS IoT 1-Click         | Manage AWS IoT devices                       | AWS IoT Core       |
| Tuya Smart / Smart Life | Smart home automation                        | Tuya Cloud         |
| Adafruit IO Dashboard   | IoT visualization                            | Adafruit IO        |

## 13. Example Workflow

1. **Device Node:** ESP8266 reads temperature and uploads to ThingSpeak Cloud.
2. **Cloud Platform:** Stores data and provides REST API.
3. **Mobile App:** ThingView fetches latest data via API.
4. **User:** Views temperature readings and sets control thresholds.

## 14. Security Considerations

- Use **HTTPS / SSL encryption** for all data exchanges.
  - Implement **OAuth2 / token-based authentication**.
  - Ensure **role-based access control** for multi-user apps.
  - Regularly update app and firmware to patch vulnerabilities.
- 

## 15. Real-World Applications

- **Smart Homes:** Control lights, appliances, and climate remotely.
  - **Industrial IoT:** Monitor equipment performance and receive fault alerts.
  - **Healthcare IoT:** View patient vitals on a doctor's mobile app.
  - **Agriculture:** Farmers track soil and weather data via mobile dashboard.
  - **Smart Cities:** Citizens get updates on parking, traffic, and pollution levels.
- 

## 16. Summary

| Aspect        | IoT Cloud Platform                 | IoT Mobile Client App              |
|---------------|------------------------------------|------------------------------------|
| Role          | Backend management, data analytics | Frontend control and visualization |
| User          | System administrator / developer   | End user / consumer                |
| Data Handling | Collects, processes, stores data   | Displays and controls data         |
| Protocols     | MQTT, CoAP, HTTP                   | MQTT, REST APIs                    |
| Examples      | AWS IoT, Azure, ThingSpeak         | Blynk, ThingView, Smart Life       |

## 17. Key Takeaways

- IoT Cloud Platforms are the **brains** managing device data and analytics.
- IoT Mobile Applications are the **hands and eyes** of the system, enabling real-time monitoring and control.
- Together, they ensure a **complete IoT solution** — from device to user.